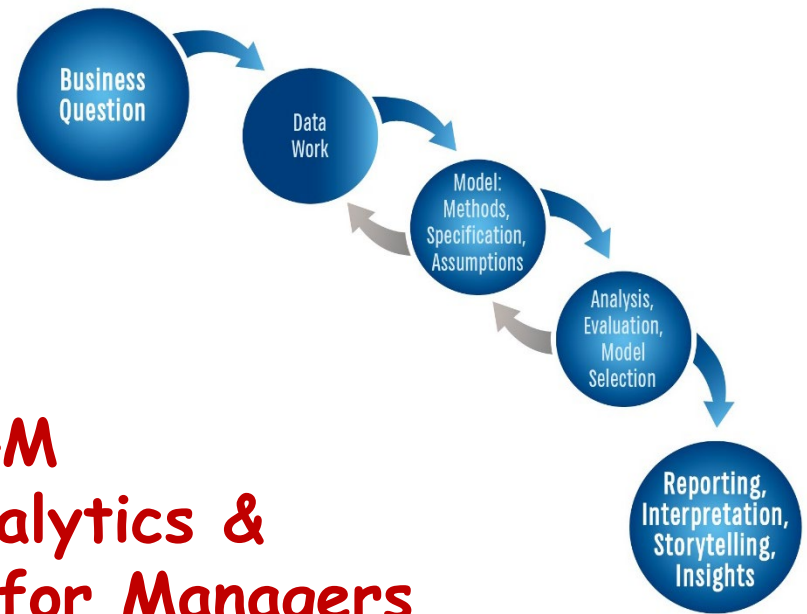




PAML4M
**Predictive Analytics &
Machine Learning for Managers**

10. Decision Tree Models

Use PAML4M_10R_DecisionTrees.Qmd

Prof. J. Alberto Espinosa
Last updated 8/1/2026



Analytics Modeling Options

	Modeling Method		
	Structured		Visual, Text, Unstructured, etc.
Descriptive	Cluster analysis, correlation, market basket analysis, sample statistics, ANOVA		Bubble charts, network diagrams, natural language processing, clustering dendograms, etc.
Predictive	Association	Decision Tree	Charts
Quantitative Value	Regression	Regression Trees	Regression plots, scatter plots, Tableau diagrams, trend charts, etc.
Classification	Logistic Regression; Other Categorical Regression Models	Classification Trees	Tree maps, interactive diagrams,
Prescriptive	Operations research, decision modeling, optimization, linear programming		Simulations, etc.

Decision Tree Issues

Decision Tree Issues

- Trees are **non-parametric**, so they are very useful when:
 - (1) **Predictive accuracy** is the main goal, rather than interpretation
 - (2) **OLS assumptions** don't hold and there is no feasible correction
- **Issues:**
 - (1) Decision trees have **limited interpretability** compared to **regression** models
 - (2) **Regression** methods often **outperform** decision **trees** in terms of predictive accuracy
 - (3) How much do we grow a tree? **mindev** doesn't answer this
- **So, why trees?**
 - (1) Use Cross-Validation (CV) to find optimal tree size
 - (2) Use advanced tree methods that train many trees to yield more stable and accurate results by aggregating results → e.g., **Bootstrap Aggregation**, **Random Forest**, **Boosted Trees**, etc.

Tree Terminology [FYI]

- **Root** Node: the top-most node (i.e., a null model with no predictors)
- **Parent** Node: the node immediately above a node
- **Ancestor** Nodes: any node above the node, either a parent or higher
- **Child** Node: a node directly below the node
- **Sibling** Nodes: nodes with the same parent node
- **Internal** or **Decision** Node: a **Splitting** node with child nodes
- **External**, **Terminal** or **Leaf** Node: a node without further splits
- **Tree Size**: total number of nodes in a tree
- **Stomp**: a tree with only one splitting node
- **Branch**, **Edge** or **Link**: a connection between two nodes
- Node **Depth**: number of links (length) between the node and the root
- **Tree Height**: the largest node depth in the tree

Growing & Pruning Trees

How Much to Grow a Tree

- Where do we **stop** the branching or segmenting?
- **Small trees** with few branches will lead to:
High bias ☹️; high train MSE ☹️; low variance 😊
The tree root (i.e., null model) is the extreme
- **Large trees with many** branches will lead to:
Low bias 😊; low train MSE 😊; overfitting ☹️; high variance ☹️
The full tree with terminal nodes is the extreme
- **CV Testing** helps identify the **optimal** tree size → the **Train MSE** decreases monotonically with tree size, but the **Test MSE** bottoms out at the optimal tree size.
- The general process is to **build** a relatively **large** regression **tree** with training data and then “**prune**” it back to a smaller “**sub-tree**” with a number of nodes/branches that **minimize** the **cross-validation ESS**

Approaches to Growing/Pruning Trees

There are a number of methods to control the size of a tree. Two popular approaches are:

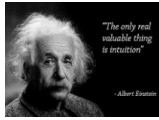
1. **Deviance Explained** → with `mindev` attribute in the `tree()` function
 - The **default** is `mindev = 0.01` → when the deviance (**MSE** or **2LL**) in a leaf is less than `0.01` or `1%` of the deviance in the **root leaf**, `tree()` stops growing the tree → further splits not worth the deviance reduction
 - $\uparrow \text{mindev} \rightarrow \downarrow \text{leaves}$ – $\downarrow \text{mindev} \rightarrow \uparrow \text{leaves}$
2. **Cross-Validation** →
 - **Fit** a relatively **large tree** with **training data** and then **prune** it back to a smaller **sub-tree** with a number of nodes/branches that **minimizes** the **CV test error**



Regression Tree Illustration

- `{library}` Topic
Search keywords
- **Decision Trees**
Optimal Regression Tree Size with Cross-Validation
Random Sample CV with Trees

Classification Trees



Classification Tree Refresher

- Classification trees work similarly to regression trees
- Except that we are predicting a classification outcome, rather than a quantitative outcome
- Often, the outcome is binary (i.e., binomial), but can also be categorical (i.e., multinomial)
- K is the number of possible classification categories
- $K = 2 \rightarrow$ binomial trees; $K > 2 \rightarrow$ multinomial trees
- The classification is based on the highest probability of classifying observation in the correct class
- Classification accuracy can be evaluated in 2 ways
 - ✓ Minimizing misclassification; or
 - ✓ Maximizing purity



Impurity Measures

- **Purity** is the extent to which classifications within terminal nodes contain data predominantly from 1 class k , as opposed to having a mixture of classes → good metric for tree splitting/pruning
- **Impurity** (opposite of purity) is a useful measure to evaluate classification outcomes in the growing or pruning of trees
- Importance for tree splitting → a tree with pure terminal leaves (i.e., impurity = 0) cannot be split further → a goal for tree growing
- Two popular **Impurity** measures:

$$\text{Gini Index} = G(m) = \sum_{k=1}^K p_{mk}(1 - p_{mk})$$

$$\text{Cross - Entropy} = D(m) = - \sum_{k=1}^K p_{mk} * \text{Log}(p_{mk})$$

- p_{mk} is the proportion of observations in class k in terminal node m
- G and D are both close to 0 when p_{mk} is close to either 0 or 1
→ i.e., higher purity → a good indication of the quality of a split



Classification Tree Illustration

- `{library}` Topic
Search keywords
- Classification Tree Size with Cross-Validation
`library(ggplot2)`
Random Split CV of Trees with the Confusion Matrix

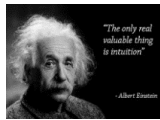


Advanced Decision Tree Methods

Overview

- Plain regression trees are easy to fit, provide **quick** predictive results, and they are **non-parametric**, so there is no **need** to check **assumptions**
- There are more **modern**, more **advanced** types of decision tree methods that often **outperform regression** in CV testing
- These methods are all based on the principle that training **many models** and **aggregating** the results yields more accurate and stable results than single models.
- These methods differ in how the training samples are generated, which predictors are specified in each training, and how the results are aggregated. We discuss 3:
 - ✓ Bootstrap Aggregation (**Bagging**) Trees
 - ✓ **Random Forest** Trees
 - ✓ **Boosted** Trees

Bootstrap Aggregation (Bagging)



Bagging Models: Intuition

- Generally, **tree models** have high **variance** and **cross-validation error** → **Bagging** aims to **reduce** these values
- In general, the **bootstrap** approach to reducing variance is based on a **simple concept**
 - If you have a data set with **n observations**, you can: (1) draw a bootstrap **random sample** of **n** observations with replacement; (2) train your model; (3) repeat this many times; and (4) **average** the results.
- Why this **works**? because the **variance** of the **sample means** is always **lower** than the **actual variance** of the raw data
- This approach is used in many statistical methods, including **trees**
- **Bagging** applies this principle to decision trees by **fitting many training sets** from the same data set and **averaging** the resulting **predictions** across all models.

Hence the name: **Bagging = Bootstrap + Aggregation !!**

Out-of-Bag (OOB) Estimation

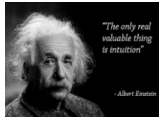
- Because bootstrap uses a sample **with replacement** from the data to do the estimation each time, the **observations not selected** in the bootstrap sample can be used for **cross-validation** testing.
- The observations not selected in the bootstrap sample are called **“out-of-bag”** observations, which can be used to compute the cross-validation **MSE**, **SSE**, **classification error**, or other statistics.
- One nice feature of Bagging trees is that one can display the **“importance”** of each variable by analyzing the **proportion** each **variable** contributes to reducing **MSE** or **error rate** across all samples.



B Agging Tree Illustrations

- `{library}` Topic
Search keywords
- **Bootstrap Aggregation (Bagging) Trees**
`library(randomForest)`
Variable Importance in BAgging trees
Bagging Tree Predictions and Cross-Validation
Classification Bagging Tree Example

Random Forest Trees



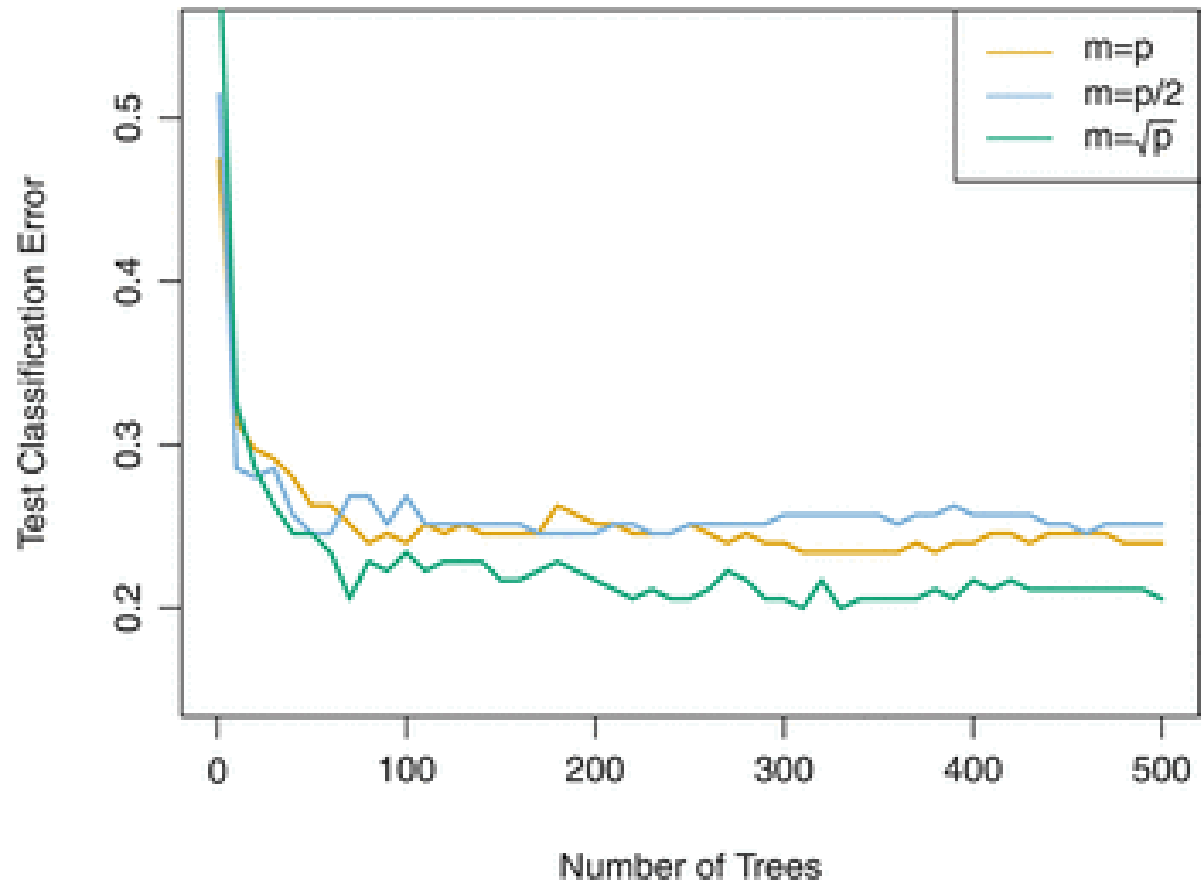
Random Forests Models: Intuition

- **Bagging** helps reduce variance, but the **trees** fitted each time have a fair amount of **correlation** with each other because they use the **same predictors**, so the trees will be somewhat **similar**
- For example, if there is one **strong predictor**, **Bagging** will often **select** this predictor at the **top** of the **tree** every time, thus generating somewhat similar models, so the **contribution** to **variance reduction** will be **small**.
- **Random Forest** overcomes using fewer **predictors** each time and selecting them **randomly** each time
- So, if there are P **predictors** in the model, **Random Forest** will only use $M < P$ of them, but because they are **randomly selected**, they will be **different** for each tree
- The number of M predictors to select is a **tuning** parameter
- **Random Forest** models are particularly useful when the number of predictors P in the model is quite **large**
- As M **approaches** P Bagging and Random become more **similar**

Random Forest Illustration: Varying M

Observations

1. $M = P \rightarrow$ same as **Bagging**
2. As the **number** of sampled **trees** increase, **error** $\downarrow$ sharply **initially** and then flattens
3. As M becomes **smaller** relative to P the test classification **error** $\downarrow$ (up to a point; needs to be tested with cross-validation)

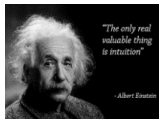


Random Forest Tree Illustrations

- `{library}` Topic
Search keywords
- Random Forest Trees
`library(randomForest)`
Random Forest Tree Predictions and Cross-Validation
Bagging Tree Predictions and Cross-Validation
Classification Bagging Tree Example

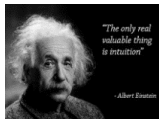


Boosted Trees



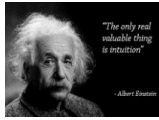
Boosted Models: Intuition

- Like **Bootstrapping**, **Boosting** is a statistical technique that can be used for **many** types of **models** → it estimates many models and **aggregates** the predictions
- **Bootstrapping** (1) generates many **random samples**, (2) training a model with each of them, and (3) averages the results across all samples
- In contrast, **Boosting** also fits many models, but does it **sequentially** with the **same sample**, one at a time → “**ensemble learning**”
 - ✓ Prior models are intentionally weakened → **weak learners**
 - ✓ Each model **improves** upon the **previous** one by predicting the **errors** (i.e., unexplained part of Y) of prior models
 - ✓ Results are then **added** up (not averaged) → “**wisdom of the crowds**”
 - ✓ There are many different boosting algorithms → **Gradient Boosting**, Adaptive Boosting (AdaBoost), eXtreme Gradient Boosting (XGBoost), etc.

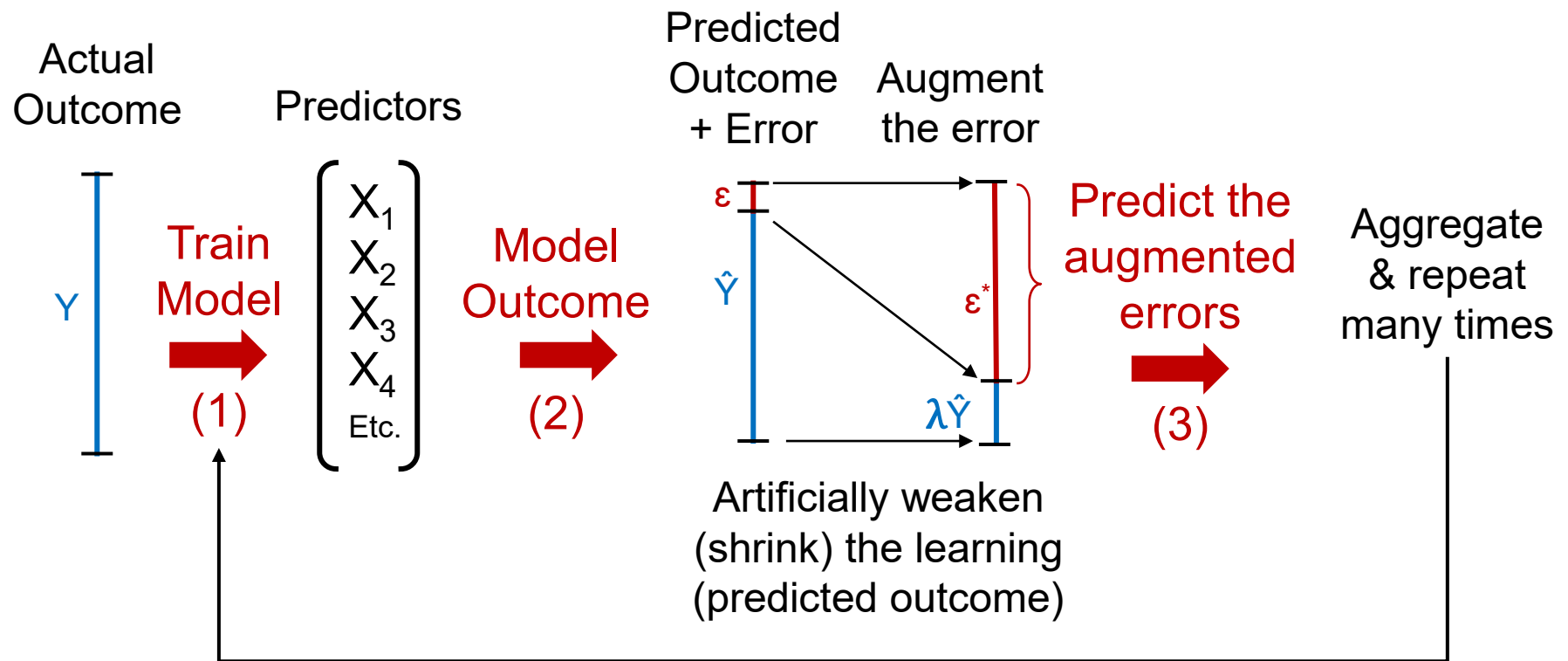


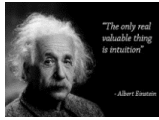
Gradient Boosting: How it Works

1. Train first model and extract the **predicted values** and **residuals**
 2. You can control the **rate of learning** of the boosted model by using only a fraction λ of the predictions, rather than the full predicted values $\rightarrow$ i.e., **shrink** the **predictions** and **expand** the **residuals**
 3. That is, **Gradient Boosting weakens** the predictions artificially by a factor of λ , thus amplifying the errors by a factor of $1 - \lambda \rightarrow$ a **weak learner**, but still better than the null model
 4. **Boosting** then trains a second model with the **same sample** to **predict** these amplified **residuals** (i.e., ϵ^* – what remains unexplained), thus **learning** from and **improving** upon the first model.
 5. It then, **adds** this prediction to the **first model**.
 6. And then, **repeats** these steps **many times** $\rightarrow$ each subsequent model **learning** from the errors of the prior models
- λ is the boosting **shrinkage** factor $\rightarrow$ a **tuning** parameter $\rightarrow$ smaller λ 's = slower rate of model learning



Gradient Boosting Illustration





Boosted Trees: Intuition

- **Bagging** fits the same tree specification many times, each with a different bootstrap sample → trees **learn fast** → **Problem**: you fit the same tree model each time, so the bootstrapped trees tend to be **similar** to each other
- **Random Forest** is like Bagging, but fits a different tree specification with each bootstrap sample → predictions often improve → trees are still learning fast
- **Gradient Boosting** → trees **learn** from **previous errors** (i.e., **residuals**) adding what we learned to the previous predictions each time → trees learn **slowly** → we can control the rate of learning with the tuning parameter λ
- **Smaller λ** → **slow** tree **learning** → less emphasis on prior model predictions, more emphasis on the unexplained outcome (i.e., the residuals).
- **Larger λ** → **faster** tree **learning** → more emphasis on what we learned in earlier model predictions

Boosted Tree Estimation

1. The **tree depth** (number of splits) is first set as a **tuning parameter**. But each tree specification will change in each iteration, so that all predictors are represented in the end.
2. Boosted trees generally work better with **low number of splits** or a **“stomp”** (i.e., only one split) → **weak learner** → often better results.
3. We estimate the **first tree** and extract the **predicted** (explained) **values** and the **residuals** (unexplained values)
4. We then **weight down** (i.e., **shrink**) the first prediction by λ (tuning parameter) to **weaken** the influence of the first prediction → which **amplifies** the **residuals** → thus controlling the **speed of learning**
5. We then estimate a **second tree** to predict the unexplained values (i.e., the **residuals**) and **add** the prediction to that of the first tree.
6. We then **repeat** this **many times**, sequentially → the **number of tree** estimates is also a **tuning parameter** → more estimates improve the accuracy but also cause **overfitting** → cross-validation can identify the optimal number of trees.

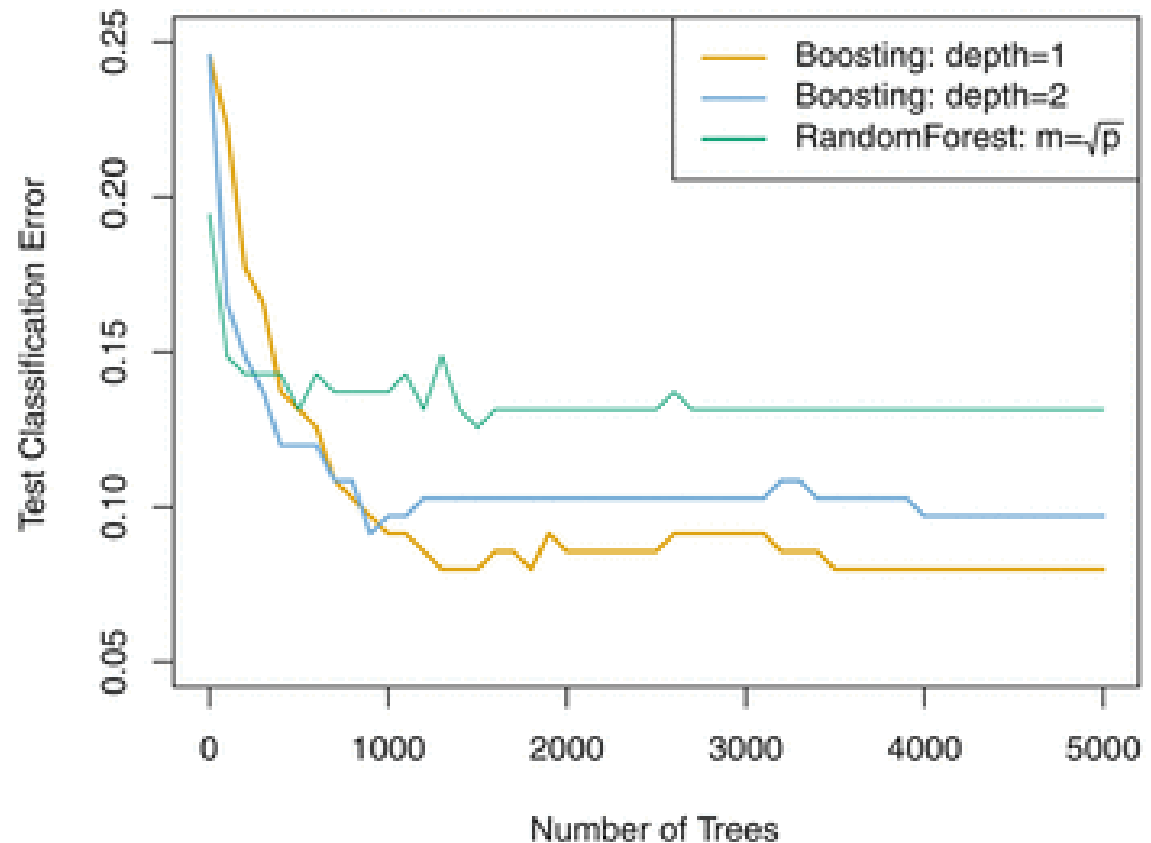
Boosted Trees: Tuning Parameters

- Boosted trees have **3** main **tuning parameters**:
 1. The number of **tree iterations B** → the number of trees (unlike Bootstrapping) improve training accuracy but also increase over-fitting
 2. The **shrinkage** parameter λ → **Small** values are better → models that **learn slowly** tend to perform **better**, but this depends on the number of tree iterations **B**
→ $\lambda = 0.01$ to 0.001 are typical
 3. **Tree depth** → i.e., number of **tree splits** (i.e., nodes) → typically small, from 1 to 6 tend to work better → **stumps** (i.e., trees with 1 split) are often the best → the predictors in the model specification change in each iteration, so there is no need to have deep trees with too many splits.

Boosting Illustration: Tuning

Observations

1. The boosted models were generated with learning shrinkage $\lambda = 0.01$
2. In this example Boosting outperformed Random Forest
3. A depth of 1 (split) outperformed a depth of 2 (splits)





Boosted Tree Illustrations

- `{library}` Topic
Search keywords
- **Boosted Trees**
`library(gbm)`
Boosted Tree Plots
Boosted Trees and Partial Dependencies
Boosted Tree Predictions and Cross-Validation
Shrinkage
Boosted Classification Tree